PDF 1 - Flask Fundamentals (10 High-Probability Interview Questions)

1. What is Flask?

Answer: Flask is a lightweight Python web framework used to build web applications and REST APIs. It provides routing, request handling, templating and extensions.

Example / Diagram:

```
from flask import Flask
app = Flask(__name__)
```

Follow-up: Why is Flask called lightweight?

2. Why is Flask called a Micro Framework?

Answer: It provides only the essential web features. Databases, authentication and forms are added through extensions.

Example / Diagram:

```
pip install flask flask-sqlalchemy flask-login
```

Follow-up: What are Flask extensions?

3. Explain app = Flask(__name__)

Answer: It creates the Flask application object. __name__ helps Flask locate templates, static files and resources.

Example / Diagram:

```
app = Flask(__name__)
```

Follow-up: Why not Flask('app')?

4. What is WSGI?

Answer: WSGI (Web Server Gateway Interface) is the standard interface between Python web applications and web servers.

Example / Diagram:

```
Browser -> Gunicorn -> Flask App
```

Follow-up: Why is Gunicorn used?

5. What is Werkzeug?

Answer: Werkzeug is the WSGI utility library that powers Flask's routing, request and response handling.

Example / Diagram:

```
from flask import request
```

Follow-up: What features does Werkzeug provide?

6. What is Jinja2?

Answer: Jinja2 is Flask's template engine used to generate dynamic HTML.

Example / Diagram:

```
return render_template('index.html',name='Sangam')
```

Follow-up: What is template inheritance?

7. Explain the Flask Request-Response Cycle.

Answer: Browser sends request -> Flask matches route -> View function executes -> Response returned.

Example / Diagram:

```
Browser
 |
 v
Flask
 |
Route
 |
Function
 |
Response
```

Follow-up: What happens if no route matches?

8. What does app.run(debug=True) do?

Answer: It starts the development server and enables automatic reload and detailed error pages.

Example / Diagram:

```
app.run(debug=True)
```

Follow-up: Should debug=True be used in production?

9. Flask vs Django

Answer: Flask is minimal and flexible. Django is full-stack with ORM, admin panel and authentication built in.

Example / Diagram:

```
# Comparison question
```

Follow-up: When would you choose Flask?

10. Development Server vs Production Server

Answer: The built-in Flask server is for development only. In production use Gunicorn/uWSGI behind Nginx.

Example / Diagram:

```
gunicorn app:app
```

Follow-up: Why not use app.run() in production?

PDF 2 - Routing & Request Handling (10 High-Probability Interview Questions)

1. What is Routing in Flask?

Answer: Routing maps a URL to a Python function using @app.route().

Code Example:

```
@app.route('/')
def home():
    return 'Home'
```

Follow-up: What happens if two routes have the same URL?

2. What is a Dynamic Route?

Answer: Dynamic routes accept variable values from the URL.

Code Example:

```
@app.route('/user/<name>')
def user(name):
    return f'Hello {name}'
```

Follow-up: What are URL converters?

3. What are URL Converters?

Answer: Converters define parameter types such as string, int, float, path and UUID.

Code Example:

```
@app.route('/post/<int:id>')
```

Follow-up: Difference between string and path converters?

4. Difference between GET and POST?

Answer: GET retrieves data; POST sends data in the request body and is used for forms and APIs.

Code Example:

```
@app.route('/login', methods=['POST'])
```

Follow-up: When should you avoid GET?

5. What is the request object?

Answer: The request object contains client data like form values, query parameters, files and JSON.

Code Example:

```
request.form
request.args
request.json
```

Follow-up: Difference between request.args and request.form?

6. request.args vs request.form

Answer: request.args reads query-string parameters; request.form reads submitted form fields.

Code Example:

```
name=request.args.get('name')
```

Follow-up: Which one is used with POST?

7. What is redirect()?

Answer: redirect() sends the client to another URL with an HTTP redirect response.

Code Example:

```
return redirect('/login')
```

Follow-up: What status code is returned?

8. What is url_for()?

Answer: url_for() generates URLs from function names instead of hardcoding paths.

Code Example:

```
url_for('home')
```

Follow-up: Why is url_for preferred?

9. How do you handle 404 errors?

Answer: Use @app.errorhandler(404) to return a custom page or JSON response.

Code Example:

```
@app.errorhandler(404)
def nf(e):
    return 'Not Found',404
```

Follow-up: What is a 500 error?

10. How do you accept multiple HTTP methods?

Answer: Specify methods in the route decorator.

Code Example:

```
@app.route('/user', methods=['GET','POST'])
```

Follow-up: What happens if a client uses an unsupported method?

PDF 3 - Templates, Forms & Sessions (10 High-Probability Interview Questions)

1. What is render_template()?

Answer: It renders an HTML template from the templates folder and allows passing Python data to HTML.

Code Example:

```
return render_template('index.html', name='Sangam')
```

Follow-up: Where does Flask look for templates?

2. What is Jinja2?

Answer: Jinja2 is Flask's template engine used to generate dynamic HTML using variables, loops and conditions.

Code Example:

```
<h1>{{ name }}</h1>
```

Follow-up: Why not write HTML directly in Python?

3. How do you pass data from Flask to HTML?

Answer: Pass keyword arguments to render_template(); access them with {{ variable }} in Jinja2.

Code Example:

```
return render_template('index.html', age=22)
```

Follow-up: Can you pass a list or dictionary?

4. Explain template inheritance.

Answer: A base template contains common layout; child templates extend it to avoid duplication.

Code Example:

```
{% extends 'base.html' %}
{% block content %}...{% endblock %}
```

Follow-up: What are template blocks?

5. How do HTML forms work with Flask?

Answer: A browser submits form data to a Flask route using GET or POST.

Code Example:

```
<form method='POST'>...</form>
```

Follow-up: Why is POST preferred for login forms?

6. How do you read form data?

Answer: Use request.form to access submitted fields.

Code Example:

```
username=request.form['username']
```

Follow-up: What is the difference between [] and get()?

7. What is Flask-WTF?

Answer: Flask-WTF simplifies forms, validation and CSRF protection.

Code Example:

```
class LoginForm(FlaskForm): ...
```

Follow-up: Why is CSRF protection important?

8. What are Sessions?

Answer: Sessions store user-specific data on the server between requests.

Code Example:

```
session['user']='Sangam'
```

Follow-up: Where is session data stored?

9. Difference between Sessions and Cookies

Answer: Sessions store data server-side; cookies store small data in the browser.

Code Example:

```
response.set_cookie('theme','dark')
```

Follow-up: Which is more secure?

10. How do you log out a user using sessions?

Answer: Remove session data using pop() or clear().

Code Example:

```
session.pop('user', None)
# or
session.clear()
```

Follow-up: What happens if the session expires?

PDF 5 - REST API, Authentication & Security (10 High-Probability Interview Questions)

1. What is a REST API?

Answer: A REST API is an architectural style where clients communicate with servers using HTTP methods and resources.

Code / Example:

```
GET /users
POST /users
```

Follow-up: What makes an API RESTful?

2. Explain HTTP methods.

Answer: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes resources.

Code / Example:

```
@app.route('/users', methods=['GET','POST'])
```

Follow-up: PUT vs PATCH?

3. What is jsonify()?

Answer: jsonify() converts Python objects into a proper JSON response with the correct MIME type.

Code / Example:

```
return jsonify({'status':'ok'})
```

Follow-up: Why use jsonify instead of str(dict)?

4. What are HTTP status codes?

Answer: Status codes indicate request results: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.

Code / Example:

```
return jsonify({'msg':'Created'}),201
```

Follow-up: 401 vs 403?

5. What is Authentication?

Answer: Authentication verifies the identity of a user before granting access.

Code / Example:

```
# Username + Password
# JWT / Session
```

Follow-up: Authentication vs Authorization?

6. What is Flask-Login?

Answer: Flask-Login manages user sessions, login state and protected routes.

Code / Example:

```
@login_required
```

```
def dashboard():
    return 'Dashboard'
```

Follow-up: How does login_required work?

7. What is JWT?

Answer: JSON Web Token is a signed token commonly used for stateless API authentication.

Code / Example:

```
Authorization: Bearer <token>
```

Follow-up: When should JWT be preferred over sessions?

8. How do you prevent SQL Injection?

Answer: Use parameterized queries or an ORM like SQLAlchemy. Never build SQL using string concatenation.

Code / Example:

```
User.query.filter_by(id=user_id).first()
```

Follow-up: Why are parameterized queries safe?

9. What is CSRF and how is it prevented?

Answer: CSRF tricks a logged-in user into sending unwanted requests. Use CSRF tokens with forms (e.g., Flask-WTF).

Code / Example:

```
{{ form.csrf_token }}
```

Follow-up: Does JWT-based API need CSRF?

10. What is CORS?

Answer: CORS controls which origins can access your API from browsers. Flask-CORS is commonly used.

Code / Example:

```
from flask_cors import CORS
CORS(app)
```

Follow-up: Why do browsers block cross-origin requests?

PDF 6 - Deployment, Coding & Scenario-Based Interviews

1. Why shouldn't app.run() be used in production?

Answer: It is a development server and is not designed for performance, security, or handling many concurrent users. Production deployments typically use Gunicorn/uWSGI behind Nginx.

Code / Diagram:

```
gunicorn app:app
```

Follow-up: What role does Nginx play?

2. What is Gunicorn?

Answer: Gunicorn is a production-grade WSGI server that runs Flask applications efficiently with multiple worker processes.

Code / Diagram:

```
gunicorn -w 4 app:app
```

Follow-up: How do worker processes improve performance?

3. Why is Nginx used with Flask?

Answer: Nginx acts as a reverse proxy, serves static files, terminates SSL, and forwards requests to Gunicorn.

Code / Diagram:

```
Client -> Nginx -> Gunicorn -> Flask
```

Follow-up: Can Gunicorn serve static files efficiently?

4. Coding: Create a dynamic route.

Answer: Create a route that accepts a username from the URL and returns a greeting.

Code / Diagram:

```
@app.route('/user/<name>')
def user(name):
    return f'Hello {name}'
```

Follow-up: What other URL converters are available?

5. Coding: Return JSON from an API.

Answer: APIs should return JSON using jsonify().

Code / Diagram:

```
from flask import jsonify
return jsonify({'success':True})
```

Follow-up: Why is jsonify preferred?

6. Scenario: User cannot access /dashboard after login.

Answer: Check session/JWT creation, login logic, protected routes, and whether authentication middleware is configured correctly.

Code / Diagram:

```
# Verify session/JWT
# Check @login_required
```

Follow-up: How would you debug this?

7. Scenario: API returns 500 Internal Server Error.

Answer: Check logs, stack trace, database connection, environment variables, and exception handling.

Code / Diagram:

```
try:
    ...
except Exception as e:
    app.logger.error(e)
```

Follow-up: What is the difference between 500 and 400?

8. Scenario: Flask app becomes slow under load.

Answer: Investigate database queries, caching, indexing, worker count, and profiling. Avoid debug mode in production.

Code / Diagram:

```
Gunicorn + Redis Cache
```

Follow-up: How would you identify the bottleneck?

9. Explain the Flask project structure.

Answer: A common structure separates routes, templates, static files, models, and configuration for maintainability.

Code / Diagram:

```
app.py
models.py
routes.py
templates/
static/
```

Follow-up: Why use Blueprints?

10. Final Interview Question: Describe a Flask project.

Answer: Explain the problem, architecture, APIs, database, authentication, deployment, challenges, and optimizations with measurable results.

Code / Diagram:

```
Problem -> Design -> API -> DB -> Security -> Deployment
```

Follow-up: What challenges did you solve and why?